

LOGICKÉ ŘÍZENÍ – PROGRAMOVACÍ JAZYKY, LOGICKÉ FUNKCE

OBSAH

1 Logické řízení	3
1.1 Logické hradlo AND – logický součin	3
1.2 Logické hradlo OR – logický součet	3
1.3 Logické hradlo XOR – exklusivní disjunkce	3
1.4 Logické hradlo NOT – negace	4
1.5 Logické hradlo NAND – negovaný součin	4
1.6 Logické hradlo NOR – negovaný součet	4
2 Mikroprocesory	5
3 PLC – Programmable logic computer	6
4 Počítače	7
5 Programovací jazyky	8
5.1 Programovací jazyky pro PLC	8
5.1.1 Instrukční list – Instruction list – IL	8
5.1.2 Sekvenční programování – Sequential Function Chart – SFC	9
5.1.3 Releová schémata – Ladder diagram – LD	9
5.1.4 Schéma funkčních bloků – Function block diagram – FBD	10
5.1.5 Strukturovaný text – Structured text – ST	10
5.2 Programovací jazyky pro mikroprocesory	11
5.2.1 Programovací jazyky C, C++, C Sharp	11
5.2.2 Programovací jazyky Java, JavaScript	12
5.2.3 Programovací jazyky Python 2, Python 3	13
Zdroje	14

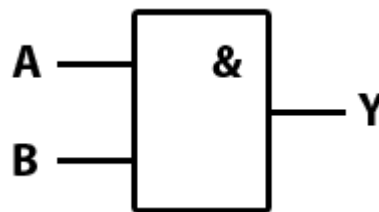
1 LOGICKÉ ŘÍZENÍ

- Logické řízení využívá logických hodnot 0 a 1 pro ovládání akčních členů.
- Logické řízení lze rozdělit na sekvenční a logické, viz maturitní okruh č. 3) Kombinační logické řízení, minimalizace; viz maturitní okruh č. 4) Sekvenční logické řízení, klopné obvody
- Logické řízení je realizováno pomocí kontaktů (vstupy), cívek (výstupy) a logických hradel (logické operace)
- Logická hradla obsahují soubor kontaktů, které tvoří funkci

1.1 Logické hradlo AND - logický součin

- Pokud jsou na vstupu hradla AND všechny vstupy v logické 1, je na výstupu tohoto hradla logická 1
- Logický součin – $0 \cdot 0 = 0$; $0 \cdot 1 = 0$; $1 \cdot 0 = 0$; $1 \cdot 1 = 1$

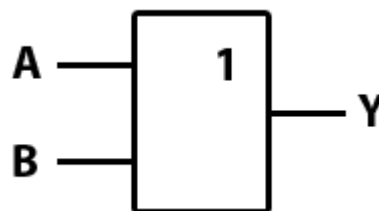
Vstup A	Vstup B	Výstup Y
0	0	0
0	1	0
1	0	0
1	1	1



1.2 Logické hradlo OR - logický součet

- Pokud je na vstupu hradla OR alespoň jedna logická 1, je výstup tohoto hradla logická 1
- Logický součet – $0 + 0 = 0$; $0 + 1 = 1$; $1 + 0 = 1$; $1 + 1 = 1$

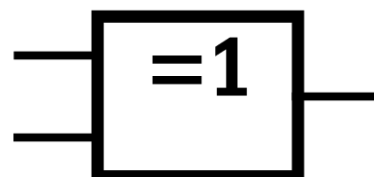
Vstup A	Vstup B	Výstup Y
0	0	0
0	1	1
1	0	1
1	1	1



1.3 Logické hradlo XOR - exklusivní disjunktce

- Pokud jsou logické hodnoty vstupu hradla XOR různé, je výstup tohoto hradla logická 1

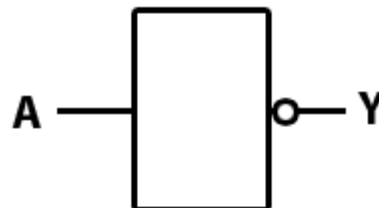
Vstup A	Vstup B	Výstup Y
0	0	0
0	1	1
1	0	1
1	1	0



1.4 Logické hradlo NOT - negace

- Pokud je na vstupu hradla NOT logická 1, je na výstupu hradla negovaná (opačná) hodnota, tedy logická 0 a naopak

Vstup A	Výstup Y
0	1
1	0

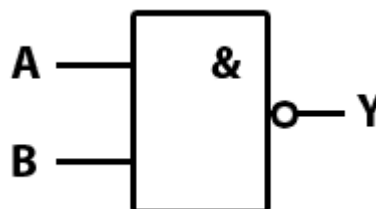


1.5 Logické hradlo NAND - negovaný součin

- Pokud jsou na vstupu hradla NAND všechny vstupy logické 1, je na výstupu tohoto hradla logická 0

- Negovaný logický součin – $0 \cdot 0 = 1$; $0 \cdot 1 = 1$; $1 \cdot 1 = 0$

Vstup A	Vstup B	Výstup Y
0	0	1
0	1	1
1	0	1
1	1	0

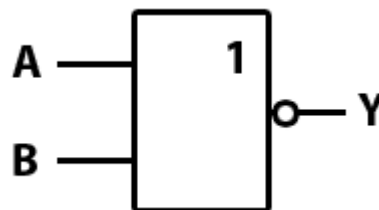


1.6 Logické hradlo NOR - negovaný součet

- Pokud je na vstupu hradla OR alespoň jedna logická 1, je výstup tohoto hradla logická 0

- Logický součet – $0 + 0 = 1$; $0 + 1 = 0$; $1 + 1 = 0$

Vstup A	Vstup B	Výstup Y
0	0	1
0	1	0
1	0	0
1	1	0



2 MIKROPROCESORY

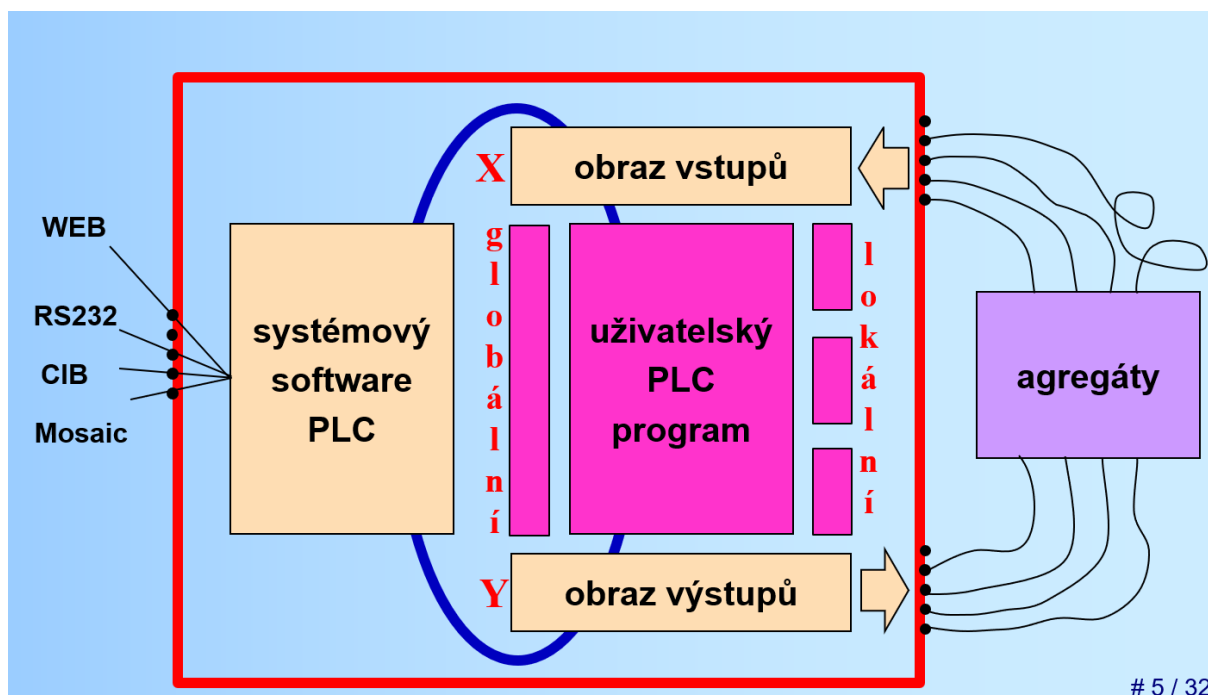
- Určené pro řízení procesů a matematické operace
- Jeden integrovaný obvod
- Levné a rychlé
- Nutnost většinou mít programátor pro programování mikroprocesorů
 - Jako programovací jazyk se využívá široké spektrum volby – C, C++, C Sharp, Python 2, Python 3, Java, Java Script, Ruby



Obrázek 1 - Mikroprocesor

3 PLC - PROGRAMMABLE LOGIC COMPUTER

- Programovatelný logický počítač
- Skládá se z několika částí – mikroprocesor, napájení, vstupy a výstupy, přídatné moduly
- Využíváno pro řízení např. automatizovaných linek
- Program v PLC probíhá pomocí smyčky (loopu), postupně proběhne systémový program PLC, zapsání výstupů, uživatelský program, sejmутí obrazu vstupů, systémový program PLC atd.



Obrázek 2 - Schéma PLC

4 POČÍTAČE

- Zařízení, které obsahuje softwarovou a hardwarovou část
- Je složen z několika komponent – mikroprocesor, paměť, úložiště, grafická karta
- Dnes jsou používány především číslicové počítače, které pracují na logice 0/1. Analogové počítače již mnoho využívány nejsou, nejznámější počítač tohoto typu je první počítač na světě ENIAC.



Obrázek 3 - Dnešní počítač

5 PROGRAMOVACÍ JAZYKY

- Programovací jazyky jsou využívány k zjednodušení programování. Bez programovacích jazyků by se muselo vše programovat pomocí strojového kódu – logická 0 a 1. Programovací jazyk obsahuje funkce (if, while, for), pro které má vytvořený strojový kód a není nutné, aby jej programátor musel znát.
- Programovací jazyky značně snižují čas potřebný na programování.

5.1 Programovací jazyky pro PLC

- Programovací jazyky pro PLC jsou stanoveny pomocí norem, tudíž jakékoliv PLC (mimo pár výjimek – Siemens) lze programovat pomocí 6 normalizovaných jazyků

5.1.1 INSTRUKČNÍ LIST - INSTRUCTION LIST - IL

- základní jazyk
- vychází z Assembleru
- nenáročné na výpočetní výkon a velikost, při větším programu se stávají nepřehlednými

```

1. LD      "Start"      ;bit processor
2. O       "Flag"
3. A       "Stop"
4. A       "BLK"
5. AN      "Error"
5. =       "Flag"
6. =       F_bB1        ;F_b to F_bB1
7. Read F_bB1          ;F_bB1 to F_b
8. L       "Time1"      ;byte processor
9. SD      "Timer1"
10. LD     "Timer1"      ;bit processor
11. =      "Start_Up"
12. L      "SP_cur"      ;byte processor
13. L      "SP_nom"
14. >=I
15. WriteF_bB          ;F_b to F_bB
16. LD     "Start_Up"    ;bit processor
17. A      F_bB          ;F_bB to F_b
18. =      "Out"
19. LD     "Start_Up"
20. =      F_bB2         ;F_b to F_bB2
21. Read F_bB2          ;F_bB2 to F_b
22. L      "Time2"      ;byte processor
23. SD     "Timer2"
24. TH     "Timer2"      ;bit processor
25. R      "Flag"
26. S      "Error"
27. A      "Stop"
28. R      "Error"

```

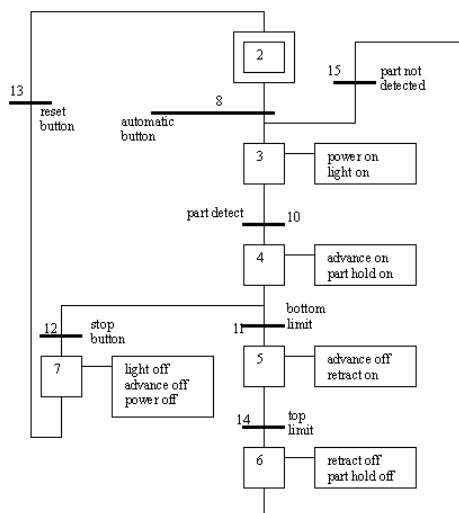
Obrázek 4 - Příklad instrukčního listu

5.1.2 SEKVENČNÍ PROGRAMOVÁNÍ - SEQUENTIAL FUNCTION CHART - SFC

- přehledný

- snadná realizace zpětných vazeb

- pro složité programy je nepřehledný, nevhodný pro datovou komunikaci a větší bloky dat



Obrázek 5 - Příklad sekvenčního programování

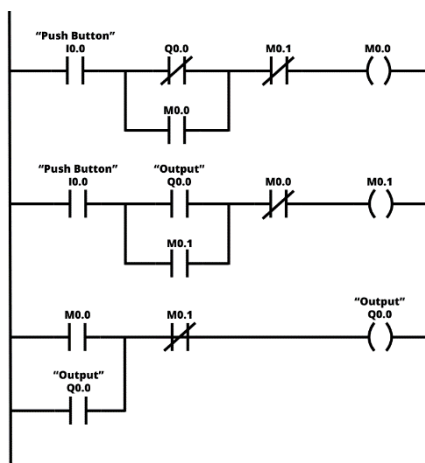
5.1.3 RELEOVÁ SCHÉMATA - LADDER DIAGRAM - LD

- vychází z logiky relé

- přehledný

- s rostoucí složitostí nabývá délka programu, přičemž je ale stále celkem přehledný

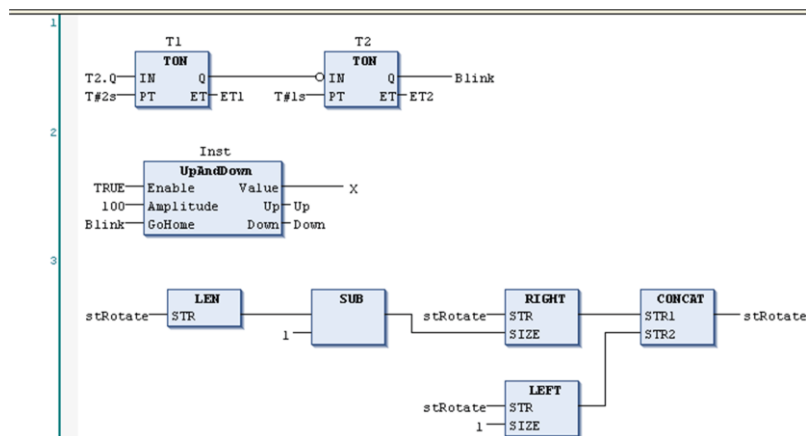
- méně vhodný pro matematické operace



Obrázek 6 - Příklad reléového schéma

5.1.4 SCHÉMA FUNKČNÍCH BLOKŮ - FUNCTION BLOCK DIAGRAM - FBD

- hradla a bloky jsou použity jako funkce
- přehledný
- nevhodný pro složité matematické operace



Obrázek 7 - Příklad funkčních bloků

5.1.5 STRUKTUROVANÝ TEXT - STRUCTURED TEXT - ST

- nutná znalost syntaxe a příkazů
- snadné matematické operace
- snadná práce s daty, komunikací atd.
- horší přehlednost
- menší přehlednost při vyšším počtu digitálních signálů

```

FindGreaterThan X
1  FUNCTION FindGreaterThan : INT
2  VAR_INPUT
3      Threshold : REAL;
4  END_VAR
5  VAR_IN_OUT
6      Values : ARRAY[1..1000] OF REAL;
7  END_VAR
8  VAR
9      Found : BOOL;
10     Index : INT;
11 END_VAR
12
13 Found := FALSE;
14 Index := 1;
15 WHILE NOT(Found) AND Index <= 1000 DO
16     IF Values[Index] > Threshold THEN
17         Found := TRUE;
18     ELSE
19         Index := Index + 1;
20     END_IF;
21 END_WHILE;
22 IF Found THEN
23     FindGreaterThan := Index;
24 ELSE
25     FindGreaterThan := 0;
26 END_IF;

```

Obrázek 8 - Příklad strukturovaného textu

5.2 Programovací jazyky pro mikroprocesory

- Pro mikroprocesory existuje mnohem větší výběr mezi programovacími jazyky.

- Programovací jazyky lze dělit na:

- 1) Imperativní – procedurální – využívají algoritmy k řešení úloh – C, C++, C Sharp, JavaScript

- 2) Funkcionální – skládají se z funkcí, někdy ani neexistují proměnné – Haskell, Miranda

- 3) Interpretované – jsou překládány za běhu programu pomocí interpreteru a rovnou prováděny – Python, PHP

- pomalé oproti kompilovaným

- 4) Kompilované – jsou přeloženy před během programu, oproti interpretovaným jsou rychlé – C, JavaScript

5.2.1 PROGRAMOVACÍ JAZYKY C, C++, C SHARP

- vyvinut pro operační systém Unix – později Linux

- kompilovaný jazyk

- imperativní programovací jazyk

- dnes je obecně známý a díky němu vzniklo mnoho dalších programů

```
#include <stdio.h>
#include <unistd.h>

int main ()
{
    printf("Running 'net join' with the following parameters: \n");
    char *domain="mydomain";
    char *user="domainjoinuser";
    char *pass="mypassword";
    char *vastool="/opt/quest/bin/vastool";
    char *ou="OU=test,DC=mtdomain,DC=local";
    char unjoin[512];

    sprintf(unjoin,"/opt/quest/bin/vastool -u %s -w '%s' unjoin -f",user,pass);

    printf("Domain: %s\n",domain);
    printf("User: %s\n",user);
    printf("-----\n");

    printf("\nUnjoin.....\n");
    system(unjoin);

    printf("\nJoin.....\n");
    execl("/opt/quest/bin/vastool", "vastool", "-u", user, "-w", pass, "join", "-c", ou,
    "-f", domain, (char*)0);
}
```

Obrázek 9 - Příklad jazyka C

5.2.2 PROGRAMOVACÍ JAZYKY JAVA, JAVASCRIPT

- dnes jeden z nejoblíbenějších jazyků
- imperativní programovací jazyk
- kompilovaný programovací jazyk
- vzhledově podobný jazyku C, liší se ale funkčně

```
import java.io.IOException;

public class TomcatEmbedded {

    private static final String EMPTY = "";

    public static void main(String... args)
        throws Exception {
        File baseFolder = new File(System.getProperty("user.dir"));
        File appsFolder = new File(baseFolder, "child: \"apps\"");

        Tomcat tomcat = new Tomcat();
        tomcat.setBaseDir(baseFolder.getAbsolutePath());
        tomcat.setPort(8080);
        tomcat.getHost().setAppBase(appsFolder.getAbsolutePath());

        // Call the connector to create the default connector.
        tomcat.getConnector();

        tomcat.addWebapp(EMPTY, docBase: ".");
        Wrapper wrapper = tomcat.addServlet(EMPTY, servletName: "hello", new HelloServlet());
        wrapper.setLoadOnStartup(1);
        wrapper.addMapping(s: "/*");

        tomcat.start();
        tomcat.getServer().await();
    }
}
```

Obrázek 10 - Příklad jazyka Java

5.2.3 PROGRAMOVACÍ JAZYKY PYTHON 2, PYTHON 3

- dnes jeden z nejoblíbenějších
- imperativní jazyk
- interpretovaný jazyk
- jednoduchá struktura
- velmi důležité dodržovat syntaxi

```

105 x += playerSpeed
106 player.setx(x)
107
108 def fire_bullet():
109     #Declare bulletState
110     global bulletState
111     if bulletState == "ready":
112         os.system("afplay laser.wav&")
113         bulletState = "fire"
114         x = player.xcor()
115         y = player.ycor()
116         bullet.setposition(x, y + 10)
117         bullet.showturtle()
118
119 def isCollision(t1, t2):
120     distance = math.sqrt(math.pow(t1.xcor()-t2.xcor(),2)+math.pow(t1.ycor()-t2.ycor(),2))
121     if distance < 15:
122         return True
123     else:
124         return False
125
126 #Create Keyboard Bindings
127 turtle.listen()
128 turtle.onkey(move_left, "Left")
129 turtle.onkey(move_right, "Right")
130 turtle.onkey(fire_bullet, "space")
131
132 #Main Game Loop
133 while True:
134     for enemy in enemies:
135         #Move the enemy
136         x = enemy.xcor()
137         x += enemySpeed
138         enemy.setx(x)
139
140         #Move the enemy back and down
141         if enemy.xcor() > 280:

```

Obrázek 11 - Příklad jazyka Python

ZDROJE